

Windows Threat Hunting & Malware Analysis

Expanded Defensive Ebook

Based on the supplied WTH Handbook, reorganized and expanded with additional defensive learning material.

Module	Topic
0	Windows Threat Hunting & Malware Analysis
1	Foundations of Windows Threat Hunting
2	Leveraging Native Windows Tools
3	Introduction to the Sysinternals Suite
4	Advanced Process Monitoring & Threat Detection
5	Persistence and Telemetry Monitoring
6	Network Forensics & Practical Incident Response
7	Memory Forensics & Rootkit Detection
8	Windows Kernel, Drivers & Advanced Rootkit Detection
9	UEFI, Bootkits & Firmware Security
10	Android Security & ADB Forensics
11	Zero-Click Exploits, Messaging Threats & Mobile Defense
12	Reverse Engineering for Defenders
13	Malware Analysis Lab & Safe Reverse Engineering
14	Threat Intelligence, YARA & Detection Engineering
15	Advanced Incident Response & Digital Forensics
16	Enterprise EDR, SIEM & Security Operations
17	Security Hardening & Prevention
18	Analyst Playbooks & Reference Workflows

Windows Threat Hunting & Malware Analysis

A modular handbook for defenders, analysts, and security practitioners.

17 chapters · 18 modules · Practical workflows

About this handbook This guide covers the full spectrum of Windows threat hunting — from native tools and Sysinternals to memory forensics, rootkit detection, mobile security, reverse engineering, and enterprise detection engineering. Each chapter is designed to be read independently or as part of the complete curriculum.

■ Contents

How to use this handbook Each chapter includes learning objectives, practical commands, investigation workflows, and analyst checklists. Use the sidebar to navigate, or read sequentially for a complete course in Windows threat hunting and malware analysis.

How to study this book

Read the modules sequentially. For each chapter, first learn the concept, then run the observation-focused exercises only on systems you own or are explicitly authorized to investigate. Keep a notebook containing hypotheses, timestamps, evidence identifiers, and conclusions.

Core evidence model

Think in six dimensions: process, file, registry/configuration, identity, network, and time. A finding becomes stronger when independent dimensions agree. A suspicious name alone is weak; a suspicious path plus unusual parent plus matching network behavior is much stronger.

Safety boundary

Malware samples, memory images, firmware assessments, and dynamic analysis belong in controlled environments. Do not test exploit behavior, persistence, or credential access against production systems or devices without explicit authorization.

Foundations of Windows Threat Hunting

Threat hunting begins with a simple idea: suspicious activity becomes easier to recognize when you understand what normal activity looks like.

Threat hunting begins with a simple idea: suspicious activity becomes easier to recognize when you understand what normal activity looks like.

Learning objectives By the end of this chapter you should be able to describe common Windows threat vectors, explain the role of scripts and C2 communications, distinguish live monitoring from post-incident forensics, and build a useful baseline for a Windows endpoint.

1.1 Understanding Threat Vectors & Script-Based Attacks

A threat vector is the route or technique through which an attacker, malicious program, or unwanted action reaches a system. On Windows endpoints, this can involve phishing attachments, malicious documents, compromised installers, browser downloads, removable media, exposed services, stolen credentials, or scripts launched through trusted interpreters.

Script-based attacks deserve special attention because the attacker may not need to introduce a traditional executable immediately. A script can act as a loader, download additional content, modify configuration, create persistence, or invoke legitimate Windows components. Common interpreters include PowerShell, Windows Script Host components, and command shells.

How a script-based intrusion can unfold

1. Initial delivery: a user receives a file, link, archive, or other content.
1. Execution: the user or another process launches the content.
1. Staging: the script may create temporary files, configuration data, or additional payloads.
1. Discovery: the activity may query processes, accounts, network configuration, or other system information.
1. Communication: the process may attempt to reach an external service for instructions or data transfer.
1. Persistence or follow-on activity: the actor may attempt to survive reboot or continue operating through another process.

For a defender, the important question is not simply “Was a script executed?” Instead, ask: Who launched it, from where, with which parent process, what did it touch, and what happened immediately afterward?

Analyst mindset: a single event rarely proves compromise. Strong findings usually come from connecting process, file, registry, user, and network evidence into a timeline.

Recognizing suspicious script activity

Useful indicators include an unusual parent-child relationship, execution from a temporary or user-writable directory, an unexpected script launched by an office application, newly created files followed by outbound connections, or a process whose name resembles a legitimate Windows component but whose path is unexpected.

These indicators should be treated as leads rather than automatic proof of maliciousness. Administrators, software updaters, management agents, and security tools can also generate unusual-looking activity.

1.1.1 Command & Control (C2)

Command and Control describes the communication channel through which an operator or malicious infrastructure can influence a compromised system. C2 traffic can be direct or hidden within ordinary-looking protocols and services.

Threat hunters therefore look beyond the existence of a network connection. Useful context includes the initiating process, destination, timing, frequency, DNS activity, remote port, executable path, user context, and whether the connection is consistent with the endpoint's normal role.

Which process opened the connection? What is the process's executable path? Who owns the process? Was the process recently created? Is the destination expected for this application? Did file or registry activity occur before the connection?

1.2 The Mindset of a Security Analyst

Threat hunting is an evidence-driven discipline. Analysts continuously form hypotheses, test them against telemetry, and revise their conclusions as new evidence appears. The objective is not to label every unusual event as malicious; it is to reduce uncertainty using reliable evidence.

Live monitoring vs. post-incident forensics

The two approaches complement one another. Live monitoring can capture transient behavior that disappears quickly, while forensic analysis can provide a broader historical timeline.

Establishing a healthy Windows baseline

A baseline is a documented picture of expected behavior. It can include normal processes, installed software, startup entries, scheduled tasks, listening services, typical network destinations, and expected resource usage.

Baselines should be role-specific. A developer workstation, domain controller, file server, and kiosk can have very different normal behavior. A good baseline therefore records context instead of relying on a universal “safe process list.”

Chapter summary

Windows threat hunting is fundamentally about context and correlation. Script execution, process creation, persistence, and network communication are individual signals; when combined with a reliable baseline and a timeline, they become much more useful evidence. The next module moves from concepts into native Windows tools that help analysts observe these signals directly.

Evidence quality ladder

Prefer direct observations over reputation labels. Record the exact command/tool, timestamp, host, user, path, PID, hash, and relevant output. Preserve the original evidence before changing the system.

Triage decision tree

Start broad enough to establish context, then narrow: host role → user → process tree → path/signature → file/registry activity → network → historical logs. Stop expanding when the evidence no longer supports the hypothesis.

Leveraging Native Windows Tools

Native Windows utilities provide a first-response view of processes, resources, scheduled activity, and system events. They are powerful because they are already present, but they must be interpreted in context.

Native Windows utilities provide a first-response view of processes, resources, scheduled activity, and system events. They are powerful because they are already present, but they must be interpreted in context.

Defensive scope: The techniques in this chapter are intended for authorized monitoring, triage, and incident response. They focus on observation rather than bypassing security controls.

2.1 Command-Line Reconnaissance with tasklist

tasklist provides a snapshot of running processes. Start with process names, PIDs, session information, and memory usage, then correlate suspicious entries with their executable path and parent process using additional tools.

```
tasklist tasklist /svc tasklist /v
```

A process name alone is weak evidence. A familiar name running from an unexpected directory, under an unexpected account, or with an unusual parent process deserves additional investigation.

2.2 Real-Time Performance & Resource Tracking

Resource Monitor (resmon) provides a useful operational view of CPU, memory, disk, and network activity. Use it to connect resource consumption with the processes responsible for it.

For network triage, examine processes with active connections and compare them with the endpoint's expected role. Unexpected outbound traffic is an investigation lead, not automatic proof of malware.

2.3 Auditing Automated Triggers with Task Scheduler

Task Scheduler is a legitimate automation system and therefore a common place to investigate persistence. Review task names, triggers, actions, run-as accounts, executable paths, and creation or modification timestamps.

Pay special attention to tasks that launch scripts or binaries from user-writable locations, have misleading names, or execute at unusual times.

2.4 Event Viewer

Event Viewer (eventvwr.msc) provides access to Windows logs. Analysts should learn the difference between Security, System, Application, and specialized operational logs.

Correlation principle: A process event becomes much more useful when its timestamp can be connected to a file creation, scheduled task, authentication event, registry modification, and network connection.

Practical workflow

1. Record the hostname, user, time, and business role.
1. Review active processes.
1. Investigate unusual process paths and relationships.
1. Review scheduled tasks and recent system events.

1. Correlate timestamps before making a conclusion.
1. Preserve evidence before remediation where possible.

Tool-selection matrix

Use Process Explorer for process relationships and metadata, ProcMon for detailed activity, Autoruns for persistence, and TCPView for live connection mapping. Combine them instead of treating any one view as complete.

Reproducibility

Record tool version, filter settings, capture interval, and analyst time zone. Another analyst should be able to understand how you reached the same observation.

Introduction to the Sysinternals Suite

The Sysinternals Suite is a collection of Windows diagnostic and security utilities. Its value comes from combining specialized views rather than treating any single tool as an oracle.

The Sysinternals Suite is a collection of Windows diagnostic and security utilities. Its value comes from combining specialized views rather than treating any single tool as an oracle.

3.1 Getting Started

Download the Suite from Microsoft's official Sysinternals distribution, extract it to an analyst-controlled directory, and keep a known-good copy. In an enterprise, record tool versions as part of the investigation.

3.2 Choosing the Right Tool

3.3 Trust and Verification

Before relying on a tool during an investigation, verify its source and integrity. Keep copies of the exact versions used so another analyst can reproduce the workflow.

3.4 Evidence Discipline

Capture screenshots or exports only when useful, preserve timestamps, record filters, and document what was observed. Avoid changing a live system unnecessarily during collection.

Important: Sysinternals tools operate within the Windows security model. A sophisticated compromise may manipulate or hide information. Always consider independent evidence sources.

Process triage framework

For every suspicious process ask: what is it, where is it, who launched it, who owns it, what privileges does it have, what did it access, and what happened next? This prevents over-reliance on process names.

False-positive control

Compare against a known-good endpoint with the same role. Legitimate software updaters, management agents, browsers, developer tools, and security products can produce unusual process trees.

Advanced Process Monitoring & Threat Detection

Process analysis becomes powerful when process identity, ancestry, executable location, signatures, loaded modules, handles, and behavior are examined together.

Process analysis becomes powerful when process identity, ancestry, executable location, signatures, loaded modules, handles, and behavior are examined together.

4.1 Process Explorer Deep Dive

Start with the process tree. Parent-child relationships often provide more context than process names. Inspect image paths, command lines, users, integrity levels, signatures, and verified publisher information where available.

4.2 Reputation and Hashing

File hashes can support reputation checks against trusted intelligence services. Treat reputation scores as evidence, not verdicts: a previously unseen file may simply be new, while a signed file can still be abused.

Privacy rule: Do not upload confidential binaries, memory dumps, or proprietary files to public services without authorization and an appropriate data-handling decision.

4.3 Process Monitor (ProcMon)

ProcMon is best used with focused filters. Capture the suspicious process, its child processes, and relevant file/registry activity around the event. Excessively broad captures can become difficult to interpret.

Investigation questions

- What files did the process create or modify?
- Which registry locations did it access?
- Did it launch another process?
- Did activity occur immediately before network communication?
- Does the observed behavior match the application's legitimate function?

4.4 Signed Does Not Mean Safe

Authenticode signatures help establish publisher identity and file integrity relative to the signed content. They do not prove that the application is being used legitimately. Analysts should combine signatures with path, parent process, command line, and behavior.

Persistence coverage

Review automatic execution across logon, services, scheduled tasks, startup locations, and application-specific mechanisms. Prioritize newly created or modified entries and entries whose paths fall outside the organization's software inventory.

Telemetry engineering

Logging should answer investigation questions without overwhelming analysts. Start with high-value events, measure storage and alert volume, then tune based on real investigations.

Persistence and Telemetry Monitoring

Persistence analysis asks how an unwanted program could regain execution after restart, logon, service changes, or other system transitions.

Persistence analysis asks how an unwanted program could regain execution after restart, logon, service changes, or other system transitions.

5.1 Autoruns

Autoruns exposes a broad set of automatic execution locations. Review entries by publisher, path, timestamp, and expected software inventory. Investigate unsigned or unexpected entries, but remember that many legitimate utilities are unsigned or uncommon.

5.2 Sysmon

Sysmon extends Windows telemetry with configurable events. A commonly used installation pattern is:

```
sysmon.exe -i -accepteula
```

In production, use a documented configuration appropriate to the environment. Excessive logging can create noise and storage pressure.

5.3 Important Sysmon Events

- Event ID 1: process creation.
- Event ID 3: network connection.
- Image load and driver-related events: useful for investigating unusual components when configured.
- File and registry telemetry: valuable for correlating execution and persistence.

5.4 Building a Detection Story

Instead of alerting on one event, build a chain: suspicious parent → unusual child → file modification → persistence change → outbound connection. The chain gives an analyst a hypothesis that can be tested.

Operational advice: document the exact Sysmon configuration and version used. Detection quality depends heavily on configuration and environment.

Network investigation model

Map process → destination → protocol → time → frequency → DNS context → endpoint role. Periodic connections deserve attention, but periodicity alone does not prove command-and-control.

Containment considerations

Before isolation, consider whether the host is a critical service, whether evidence is volatile, and whether the suspected activity could spread. Follow the organization's incident-response plan.

Network Forensics & Practical Incident Response

Network evidence can expose behavior that endpoint tools fail to explain. The goal is to understand which process communicated, where it connected, when it connected, and whether the communication fits the system's role.

Network evidence can expose behavior that endpoint tools fail to explain. The goal is to understand which process communicated, where it connected, when it connected, and whether the communication fits the system's role.

6.1 TCPView

TCPView provides a live view of TCP and UDP endpoints. Use it to map active connections to processes and identify unexpected destinations or listening services.

6.2 Practical Suspicious-File Workflow

1. Do not execute an unknown file on a production endpoint simply to “see what happens.”

1. Record file name, path, hash, source, timestamps, and user context.

1. Determine whether the file is expected for the installed application.

1. If dynamic analysis is authorized, use an isolated laboratory or sandbox.

1. Observe process creation, file changes, registry changes, and network activity.

1. Correlate telemetry into a timeline.

1. Contain according to incident-response procedures.

1. Preserve relevant evidence before cleanup where feasible.

6.3 Network Indicators

Useful indicators include unusual destinations, periodic beacon-like timing, unexpected protocols, newly observed domains, abnormal DNS behavior, and a process communicating in a way inconsistent with its function.

6.4 Incident Response

6.5 Reporting

A useful incident report separates observed facts from analyst interpretation. Include timestamps, evidence identifiers, affected assets, confidence, actions taken, and unresolved questions.

Memory-analysis discipline

Treat a memory image as evidence. Hash it, preserve the original, work from a copy, document acquisition details, and compare multiple enumeration views when possible.

Rootkit reasoning

A mismatch between two views can indicate manipulation, but it can also result from legitimate differences in enumeration methods. Validate with independent evidence such as logs, trusted boot state, and endpoint telemetry.

Memory Forensics & Rootkit Detection

When an endpoint's normal views may be incomplete, memory forensics provides an independent source of evidence. It is particularly useful for investigating injected memory, hidden processes, unusual drivers, and artifacts that do not appear in ordinary process listings.

When an endpoint's normal views may be incomplete, memory forensics provides an independent source of evidence. It is particularly useful for investigating injected memory, hidden processes, unusual drivers, and artifacts that do not appear in ordinary process listings.

7.1 Why Memory Matters

Running programs and operating-system state exist in volatile memory. A disk-only investigation can miss transient execution or in-memory modifications.

7.2 Volatility Workflow

Acquire memory using an approved collection method, preserve the image, hash it, and analyze a copy. Framework plugins can enumerate processes, modules, network artifacts, handles, and other structures. Comparing independent enumeration methods can reveal inconsistencies.

Evidence principle: memory acquisition is sensitive to system state. Record acquisition time, tool version, host details, and chain-of-custody information.

7.3 Rootkits and DKOM

Kernel-level malware may attempt to manipulate operating-system structures. Analysts can look for discrepancies between different views of process, module, or object state. A discrepancy is a lead requiring validation, not proof by itself.

7.4 Defensive Layers

Use multiple sources: memory, event telemetry, trusted boot measurements, endpoint security, network telemetry, and offline analysis. The goal is resilience against manipulation of any one view.

Kernel trust model

Kernel-mode components have broad privileges, so driver provenance, signing, installation history, and expected inventory matter. Use memory and boot evidence when ordinary endpoint views are suspect.

Layered defense

VBS, HVCI, Secure Boot, endpoint telemetry, least privilege, and application control reduce the impact of low-level compromise. Defense should assume individual sensors can fail.

Windows Kernel, Drivers & Advanced Rootkit Detection

Kernel security requires understanding the boundary between user mode and kernel mode, drivers, callbacks, memory integrity, and trusted boot. This chapter stays focused on defensive analysis rather than instructions for building or deploying rootkits.

Kernel security requires understanding the boundary between user mode and kernel mode, drivers, callbacks, memory integrity, and trusted boot. This chapter stays focused on defensive analysis rather than instructions for building or deploying rootkits.

8.1 User Mode vs Kernel Mode

User-mode applications operate with restricted privileges. Kernel-mode components, including drivers, have significantly greater access and therefore a much larger impact if compromised.

8.2 Driver Triage

Investigate driver names, paths, publishers, signatures, service configuration, load times, and relationships to recently installed software. Unexpected kernel components deserve careful validation.

8.3 ETW and Kernel Telemetry

Windows provides multiple telemetry channels. Enterprise EDR products may combine ETW, kernel callbacks, protected components, sensor drivers, and cloud analytics. No single telemetry source should be considered infallible.

8.4 VBS, HVCI and Trusted Computing

Virtualization-based security can isolate sensitive security functions from ordinary kernel execution. Hardware-backed protections and Secure Boot also raise the cost of persistent low-level compromise.

8.5 Detection Strategy

- Compare expected driver inventory with observed state.
- Validate signatures and provenance.
- Correlate driver activity with installation events.
- Use memory analysis when kernel manipulation is suspected.
- Escalate suspected firmware compromise to specialist response teams.

Trusted-boot investigation

When firmware or boot compromise is suspected, document firmware version, Secure Boot state, platform configuration, trusted measurements where available, and vendor-supported recovery paths.

Escalation rule

Firmware incidents are high-impact and platform-specific. Preserve evidence and involve specialist responders rather than experimenting with unverified recovery procedures.

UEFI, Bootkits & Firmware Security

Some threats operate before the operating system is fully running. Defenders therefore need a trusted-boot model that extends below Windows itself.

Some threats operate before the operating system is fully running. Defenders therefore need a trusted-boot model that extends below Windows itself.

9.1 Secure Boot

Secure Boot establishes a chain of trust for boot components. Organizations should monitor its state and investigate unexpected changes to firmware or boot configuration.

9.2 Firmware Risk

Firmware compromise is uncommon but high impact. Evidence can involve firmware versions, configuration changes, platform security settings, and vendor-specific integrity mechanisms.

9.3 CHIPSEC and Hardware Assessment

CHIPSEC is a platform security assessment framework useful for authorized hardware and firmware checks. Run such assessments under documented procedures and interpret findings with platform-specific knowledge.

9.4 Recovery

If firmware compromise is credible, normal malware cleanup may be insufficient. Follow the hardware vendor's trusted recovery process, re-establish firmware trust, rotate relevant credentials where appropriate, and validate the boot chain afterward.

Android evidence model

Record device model/build, security configuration, installed packages, permissions, signing information, relevant logs, network state, and acquisition conditions. Android version and device policy can materially change what is available.

Mobile lab practice

Use dedicated test devices, controlled accounts, isolated networks, and reproducible snapshots where supported. Never use another person's device for forensic experimentation without authorization.

Android Security & ADB Forensics

Android investigations require a different evidence model: applications run inside a permission and sandbox architecture, while ADB can provide authorized diagnostic access to devices prepared for development or enterprise investigation.

Android investigations require a different evidence model: applications run inside a permission and sandbox architecture, while ADB can provide authorized diagnostic access to devices prepared for development or enterprise investigation.

10.1 ADB Basics

```
adb devices adb shell getprop adb shell pm list packages adb shell dumpsys activity activities  
adb shell dumpsys package
```

These commands help establish device identity, configuration, installed packages, and activity state. Availability varies by Android version and device policy.

10.2 Android Evidence

Potential evidence includes application metadata, logs, package installation history, permissions, network state, account configuration, and device security settings. Access to protected application data depends on Android version, device state, authorization, and acquisition method.

10.3 Mobile Malware Analysis

For authorized analysis, combine static inspection of APK metadata and code with controlled dynamic observation. Record package name, signing certificate, permissions, components, network endpoints, and behavioral observations.

Do not bypass device security or access another person's device without authorization. Mobile forensics must follow applicable law, consent, and organizational procedures.

10.4 Mobile Hardening

Keep Android updated, use strong screen-lock protection, enable platform security features, install applications from trusted sources, review permissions, and use enterprise management where appropriate.

Zero-click defensive model

Focus on reducing exposure: rapid patching, secure device configuration, application updates, mobile threat defense where appropriate, and preservation of evidence when unexplained crashes or anomalies appear.

Incident indicators

A single crash is not proof of exploitation. Look for correlated application restarts, unexplained network behavior, configuration changes, and other independent signals.

Zero-Click Exploits, Messaging Threats & Mobile Defense

Modern messaging platforms have faced highly sophisticated vulnerabilities in which malicious content can trigger processing without an obvious user interaction. This chapter explains the defensive lifecycle without providing exploit construction or weaponization instructions.

Modern messaging platforms have faced highly sophisticated vulnerabilities in which malicious content can trigger processing without an obvious user interaction. This chapter explains the defensive lifecycle without providing exploit construction or weaponization instructions.

11.1 What “Zero-Click” Means

A zero-click vulnerability is generally one where exploitation can occur with little or no deliberate user interaction. The vulnerable component may automatically parse messages, media, calls, documents, or other content.

11.2 Why These Attacks Are Difficult

Modern applications process complex formats automatically. A flaw in a parser, decoder, memory-management routine, or IPC boundary can therefore become security-relevant before a user consciously opens content.

11.3 Defensive Detection

- Keep the operating system and messaging application fully patched.
- Monitor vendor security advisories and emergency updates.
- Use mobile threat-defense or enterprise telemetry where appropriate.
- Investigate unexplained crashes, battery/network anomalies, and unexpected application restarts.
- Preserve device evidence before resetting a suspected compromised device.

11.4 If Compromise Is Suspected

Isolate the device according to incident-response guidance, avoid deleting evidence unnecessarily, document the time and symptoms, notify the responsible security team, and use trusted recovery procedures. High-risk targets may require specialist mobile forensic analysis.

Important: Security research should use intentionally vulnerable test environments, vendor programs, or authorized laboratories. Do not test exploit techniques against real people's devices or production services.

RE workflow

Move from identification to structure to behavior: hash and identify the file, inspect metadata and PE structure, review strings/imports, then correlate static observations with controlled dynamic behavior.

Analyst notes

When reversing, keep a running glossary of function names, hypotheses, and confidence. Rename functions only when the evidence supports the interpretation; record uncertain conclusions as hypotheses.

Reverse Engineering for Defenders

Reverse engineering turns a compiled artifact into evidence. The objective of defensive RE is to understand what a program is designed to do, how it behaves, and whether that behavior is consistent with its stated purpose.

Reverse engineering turns a compiled artifact into evidence. The objective of defensive RE is to understand what a program is designed to do, how it behaves, and whether that behavior is consistent with its stated purpose.

12.1 Static Analysis

Begin with file type, architecture, hashes, metadata, strings, imports, exports, embedded resources, and signatures. Tools such as Detect It Easy, PE-bear, Ghidra, and similar analyzers can help identify structure.

12.2 Dynamic Analysis

Execute only in an isolated, authorized laboratory. Observe process trees, file changes, registry activity, network communication, mutexes or synchronization objects, and persistence behavior.

12.3 Ghidra Workflow

1. Import the sample into an isolated project.
1. Identify architecture and entry points.
1. Review strings and imported APIs.
1. Use the decompiler to understand relevant functions.
1. Rename functions and variables as understanding improves.
1. Correlate static findings with observed runtime behavior.

12.4 PE Structure

Windows Portable Executable files contain headers, sections, imports, exports, resources, and other metadata. Abnormal section names, permissions, overlays, or unusual import patterns can be useful investigation leads.

12.5 Anti-Analysis

Malware may detect debuggers, sandboxes, or analysis artifacts. From a defender's perspective, treat anti-analysis behavior as a finding and use multiple independent observations rather than attempting to defeat protections on a real target.

Lab checklist

Dedicated environment → restricted networking → controlled DNS → snapshots → sample hashing → observation plan → evidence preservation → cleanup/revert. A VM is a risk-reduction layer, not a guarantee of safety.

Reporting standard

Report behavior, indicators, confidence, affected artifacts, and defensive actions. Avoid publishing operational exploit or credential material.

Malware Analysis Lab & Safe Reverse Engineering

A professional malware-analysis lab separates research from production. Isolation, repeatability, evidence preservation, and controlled observation are more important than collecting the largest number of tools.

A professional malware-analysis lab separates research from production. Isolation, repeatability, evidence preservation, and controlled observation are more important than collecting the largest number of tools.

13.1 Lab Architecture

Use dedicated virtual machines or hardware, snapshots, restricted networking, controlled DNS, and separate storage. Never assume that a virtual machine alone makes a malicious sample safe.

13.2 Static-to-Dynamic Pipeline

13.3 Evidence

Hash samples before and after analysis, keep notes, record tool versions, preserve screenshots only where useful, and maintain a clear chain of custody for incident-related evidence.

13.4 Safe Reporting

Document behavior, indicators, confidence, and defensive recommendations. Avoid publishing operational exploit details or live credentials. Where appropriate, share hashes and indicators through trusted intelligence channels.

Detection engineering principles

Prefer stable behavior and context over brittle single indicators. Test detections against representative benign activity and measure false positives before deployment.

Detection lifecycle

Hypothesis → telemetry mapping → minimal rule → validation → tuning → documentation → periodic review. Every detection should have an owner and a reason for existing.

Threat Intelligence, YARA & Detection Engineering

Threat intelligence turns individual observations into reusable detections. Good detection engineering emphasizes durable behavior and context rather than brittle indicators alone.

Threat intelligence turns individual observations into reusable detections. Good detection engineering emphasizes durable behavior and context rather than brittle indicators alone.

14.1 Indicators

Hashes, domains, IP addresses, filenames, registry paths, and certificates can be useful, but they change. Behavioral and contextual detections often remain valuable longer.

14.2 YARA

YARA rules can classify files or memory content based on patterns. Build rules from stable characteristics and test them against both known positives and representative benign samples.

14.3 Sigma and Log Detection

Sigma-style detections express log-based behavior in a platform-neutral form. Focus on meaningful combinations such as suspicious parent-child relationships, unusual command interpreters, or unexpected persistence changes.

14.4 Detection Lifecycle

1. Define the threat hypothesis.
1. Identify available telemetry.
1. Write a minimal detection.
1. Test against benign activity.
1. Measure false positives.
1. Add context and tuning.
1. Document and review periodically.

IR timeline template

For each important event record timestamp, host, user, source, action, evidence identifier, and confidence. Separate observed facts from interpretations so later analysts can challenge assumptions.

Recovery validation

Recovery is complete only after trusted state is restored, persistence is checked, credentials are addressed as required, and monitoring confirms that the original behavior does not recur.

Advanced Incident Response & Digital Forensics

Incident response is a controlled process for understanding, containing, eradicating, and learning from security events.

Incident response is a controlled process for understanding, containing, eradicating, and learning from security events.

15.1 Preparation

Maintain logging, asset inventories, escalation contacts, backup strategies, forensic procedures, and tested recovery plans before an incident occurs.

15.2 Identification

Validate the alert, establish scope, identify affected accounts and assets, and create a preliminary timeline.

15.3 Containment

Choose short-term and long-term containment appropriate to the incident. Consider business impact, evidence preservation, and the possibility of attacker lateral movement.

15.4 Eradication and Recovery

Remove persistence, remediate vulnerabilities, reset compromised credentials where required, restore trusted systems, and monitor closely.

15.5 Lessons Learned

Determine why controls failed, which telemetry was missing, and what engineering changes will reduce recurrence.

Golden rule: containment is not the same as eradication. A disconnected endpoint can still contain evidence of compromise or persistence.

SOC investigation layers

Alert → enrichment → scope → timeline → containment decision → evidence collection → remediation → validation.
Good operations reduce analyst guesswork through normalized identity, asset context, and reliable telemetry.

Purple-team measurement

A useful exercise measures whether telemetry appeared, whether the detection fired, whether the alert was understandable, and whether the response playbook worked—not merely whether a simulation succeeded.

Enterprise EDR, SIEM & Security Operations

Enterprise defense depends on collecting signals from endpoints, identities, cloud services, applications, and networks, then turning them into prioritized investigations.

Enterprise defense depends on collecting signals from endpoints, identities, cloud services, applications, and networks, then turning them into prioritized investigations.

16.1 EDR

Endpoint Detection and Response products collect endpoint telemetry and provide investigation and response capabilities. Their effectiveness depends on sensor health, configuration, coverage, and analyst interpretation.

16.2 SIEM

A SIEM correlates logs from many sources. Good normalization and asset identity are critical to producing useful investigations.

16.3 SOC Workflow

1. Alert triage.
1. Enrichment.
1. Scope assessment.
1. Containment decision.
1. Evidence collection.
1. Remediation.
1. Post-incident review.

16.4 Purple Teaming

Defenders can validate detections through controlled simulations in authorized environments. Measure whether expected telemetry appeared and whether analysts could investigate it.

Hardening priority order

Start with patching and asset visibility, then identity and privilege controls, application control, endpoint protection, network segmentation, telemetry, backups, and tested response procedures.

Resilience principle

Prevention, detection, response, and recovery should reinforce one another. A control that cannot be monitored or recovered from is difficult to trust operationally.

Security Hardening & Prevention

The best investigation is the one an organization never needs to perform. Hardening reduces attack surface and increases the cost of successful intrusion.

The best investigation is the one an organization never needs to perform. Hardening reduces attack surface and increases the cost of successful intrusion.

17.1 Windows

- Maintain security updates.
- Use least privilege and application control.
- Enable modern endpoint protection and tamper resistance.
- Protect administrative accounts.
- Collect high-value telemetry.

17.2 Android

- Keep devices and apps patched.
- Use strong device authentication.
- Review application permissions.
- Use managed-device controls for enterprise fleets.

17.3 Network

- Segment sensitive systems.
- Monitor egress traffic.
- Use secure DNS and appropriate filtering.
- Restrict administrative protocols.

17.4 People and Process

Security awareness, incident playbooks, access reviews, vendor risk management, and tested backups are technical controls' essential partners.

Playbook quality test

A playbook should specify trigger, scope, evidence to collect, decision points, containment options, escalation criteria, recovery validation, and closure requirements.

Final analyst habit

Write down the hypothesis before collecting evidence, then update it as evidence changes. This reduces confirmation bias and makes investigations easier to review.

Analyst Playbooks & Reference Workflows

This chapter turns the handbook into repeatable operational playbooks.

This chapter turns the handbook into repeatable operational playbooks.

18.1 Suspicious Windows Process

1. Capture process identity and path.
1. Inspect parent-child relationship.
1. Check signature and hash.
1. Review file and registry activity.
1. Review network connections.
1. Correlate with Sysmon and Windows logs.
1. Assess scope and contain if required.

18.2 Suspicious Android Application

1. Record package and signing information.
1. Review permissions and components.
1. Check device telemetry.
1. Use an authorized lab for dynamic analysis.
1. Preserve evidence and escalate if compromise is suspected.

18.3 Suspected Firmware Compromise

1. Escalate to specialist responders.
1. Record platform and firmware state.
1. Validate Secure Boot and trusted measurements.
1. Use vendor-supported recovery procedures.
1. Re-establish trust before returning the system to service.

18.4 Final Analyst Checklist

Appendix — Quick Analyst Reference

Process triage

Identity → path → parent → signature/hash → privileges → file/registry activity → network → timeline.

Persistence triage

Startup locations → services → scheduled tasks → application-specific autoruns → timestamps → provenance.

Network triage

Process → destination → DNS → port/protocol → timing/frequency → expected role → historical context.

Memory triage

Acquire → hash → preserve original → analyze copy → enumerate independently → validate discrepancies.

Malware lab

Isolate → snapshot → hash → static analysis → controlled dynamic analysis → collect evidence → revert.

Detection engineering

Hypothesis → telemetry → minimal rule → benign testing → false-positive tuning → deployment → review.

Incident response

Prepare → identify → contain → eradicate → recover → validate → lessons learned.